

长上下文端到端测试时训练

Arnav Tandon^{*1,3}, Karan Dalal^{*1,4}, Xinhao Li^{*5}, Daniel Kocaja^{*3}, Marcel Rød^{*3}, Sam Buchanan⁴,
Xiaolong Wang⁵, Jure Leskovec³, Sanmi Koyejo³, Tatsunori Hashimoto³, Carlos Guestrin³,
Jed McCaleb¹, Yejin Choi², Yu Sun^{*2,3}
¹ Astera Institute ² NVIDIA ³ Stanford University ⁴ UC Berkeley ⁵ UC San Diego

摘要

本文将长上下文语言建模表述为持续学习问题，而非架构设计问题。在此表述下，我们仅使用标准架构——具有滑动窗口注意力的变换器（Transformer）。然而，我们的模型在测试时通过给定上下文上的下一词元预测持续学习，将其读取的上下文压缩到其权重中。此外，我们在训练时通过元学习改进模型在测试时学习的初始化。总体而言，我们的方法作为测试时训练（Test-Time Training, TTT）的一种形式，在测试时（通过下一词元预测）和训练时（通过元学习）均为端到端（End-to-End, E2E），与以往形式形成对比。我们进行了大量实验，重点关注扩展特性。特别是，对于使用 1640 亿词元训练的 30 亿参数模型，我们的方法（TTT-E2E）在上下文长度上的扩展方式与具有全注意力的变换器相同，而其他方法（如 Mamba 2 和门控 DeltaNet）则不然。然而，与循环神经网络（RNN）类似，TTT-E2E 的推理延迟与上下文长度无关，使其在 128K 上下文上比全注意力快 2.7× 倍。我们的代码已公开。

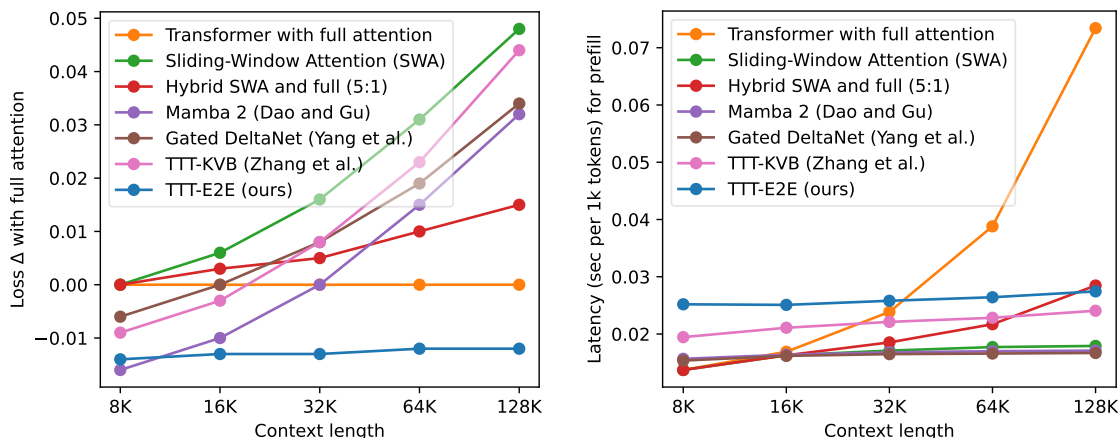


图 1. 在测试损失（左）和延迟（右）方面随上下文长度的扩展。左：我们的方法（TTT-E2E）在 128K 上下文长度上将最差曲线（绿色）变为最佳曲线（蓝色）。损失 $\Delta(\downarrow)$, (y 值) 计算为（所报告方法的损失） - （具有全注意力的变换器的损失），因此全注意力本身的损失 Δ of $y=0$ 处的平坦曲线。其他方法在更长上下文下产生更差的损失 Δ in, 而 TTT-E2E 保持了对全注意力的相同优势。所有模型均有 30 亿参数，并使用 1640 亿词元训练。右：与滑动窗口注意力（SWA）和 RNN 基线类似，TTT-E2E 的推理延迟与上下文长度无关，使其在 H100 上对 128K 上下文比全注意力快 2.7× 倍。

*Core 位贡献者。贡献声明见参考文献之前。通讯作者：arnuv@stanford.edu, kdalal@berkeley.edu, yusun@cs.stanford.edu。

1 引言

人类能够在一生中通过更多经验提升自己，尽管他们对确切细节的回忆并不完美。想想你的第一堂机器学习课：你可能不记得讲师在课堂上的第一个词，但你学到的直觉可能正在帮助你理解本文，即使那堂课发生在多年前。

另一方面，具有自注意力的变换器（Transformer）仍然难以高效处理相当于人类多年经验的长期上下文，部分原因在于它们被设计用于近乎无损的召回。对整个上下文的自注意力，也称为全注意力，必须为每个新标记扫描所有先前标记的键和值。因此，它容易关注到每个细节，但每个标记的成本随上下文长度线性增长，并迅速变得难以承受。

作为变换器的替代方案，循环神经网络（RNN）如曼巴 2（Mamba 2）[32] 和门控德尔塔网络（Gated DeltaNet）[104] 每个标记的成本恒定，但在较长上下文中效果变差，如图 1 所示。一些现代架构用滑动窗口 [1, 107] 近似全注意力，或将注意力层与循环神经网络层堆叠 [91, 11]。然而，这些技术在利用较长上下文提升语言建模性能方面仍不如全注意力有效。

我们如何设计一种每个标记成本恒定的有效语言建模方法？具体来说，如何在不回忆每个细节的情况下，在较长上下文中实现更好的性能，如开篇例子所示？关键机制是压缩。例如，人类将大量经验压缩进大脑，保留重要信息而忽略许多细节。对于语言模型，通过下一标记预测进行训练也将大量数据压缩到其权重中。那么，如果在测试时对给定上下文继续进行下一标记预测训练，会怎样？

这种形式的测试时训练（Test-Time Training, TTT），类似于一种称为动态评估 [72, 60] 的旧思想，仍缺少一个环节：在训练时，我们优化模型开箱即用的损失，而不是测试时训练后的损失。为解决这种不匹配，我们通过元学习 [38, 79, 58] 而非标准预训练来准备模型用于测试时训练的初始化。具体来说，每个训练序列首先被视为测试序列，因此我们在内循环中对其进行测试时训练。然后，我们对许多独立训练序列在测试时训练后的损失取平均，并在外循环中通过梯度的梯度 [71, 3, 27] 优化该平均值相对于模型测试时训练初始化的参数。

总之，我们的方法在两个方面是端到端的。我们的内循环直接优化下一标记

网络末端的预测损失，与先前关于长上下文测试时训练（TTT）的工作形成对比 [86, 110]；第 2.4 小节通过我们方法的另一种推导解释了这一差异。此外，我们的外层循环直接优化测试时训练后的最终损失，与动态评估 [72, 60] 形成对比，如前所述。我们的关键结果在图 1 中突出显示，其余结果在第 3 节中呈现。

测试时训练的概念框架有着悠久的历史，在长上下文之外有许多应用，并且有许多没有元学习的形式 [85, 12, 45, 2]。我们的工作也受到快速权重文献 [38, 79, 77, 49] 的启发，尤其是 Clark 等人的 [17]，他们与我们有着相同的高层方法。第 4 节详细讨论了相关工作。

2 方法

考虑下一个词元预测的标准任务，在测试时包含两个阶段：• 预填充：以 $T+1$ 个给定词元为条件 $x_0, x_1, \dots, x_T$ ，其中 x_0 是序列开始（<BOS>）词元。

- 解码：预测下一个词元所有可能实例上的分布 $\hat{p}_{T+1}$ 。

测试损失为 $\text{CE}(\hat{p}_{T+1}, x_{T+1})$ ，其中 CE 是交叉熵， x_{T+1} 由自然生成。

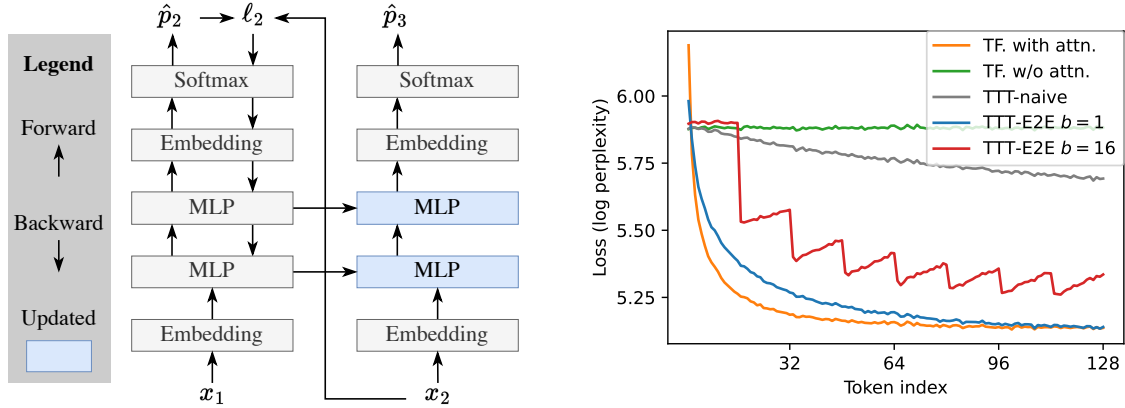


图 2. 玩具示例。左：给定 x_1 和 x_2 作为上下文，我们希望预测未知的 x_3 。我们的玩具基线是一个没有自注意力的变换器（仅使用向上的箭头），实际上是一个二元模型，因为它没有关于 x_1 的记忆。测试时训练（使用所有箭头）首先尝试从 x_1 预测 x_2 作为练习：它计算 x_2 与预测 $\hat{p}_2$ 之间的损失 ℓ_2 ，然后对 ℓ_2 执行一步梯度下降。现在 x_1 的信息存储在更新后的多层感知机（蓝色）中。右：玩具示例中各种方法的词元级测试损失 ℓ_t ，如第 2.2 小节所述，除了第 2.3 小节讨论的测试时训练端到端 $b = 16$ 。特别是，测试时训练端到端 $b = 1$ 将绿线（我们的玩具基线）变为蓝线，其性能几乎与橙色（使用全注意力）一样好。

为便于阐述，我们首先关注预填充 $T+1$ 个词元然后解码单个词元的任务。在此设置下，对完整上下文的自注意力，也称为全注意力，预填充的计算复杂度为 $O(T^2)$ ，解码的计算复杂度为 $O(T)$ 。我们现在讨论使用测试时训练（TTT）的方法，其预填充复杂度为 $O(T)$ ，解码复杂度为 $O(1)$ 。

2.1 通过下一词元预测的测试时训练

为引出本文的主要方法，我们引入一个示例，该示例将一直发展到第 2.3 小节的中部。该示例基于一个相当简单的架构：一个移除了所有自注意力层的变换器，仅保留多层感知机层。我们的示例基线——轻率地将此架构应用于语言建模——实际上是一个二元模型，因为它没有对先前词元的记忆。我们的目标是孤立地理解测试时训练的效果，而不受其他序列建模组件的干扰。

赋予基线架构记忆的一种方法是在上下文上训练它。与标准预训练类似，我们可以预测 $\hat{p}_t$ 并在每个 $t = 1, \dots, T$ 处将其与 x_t 进行比较，作为练习。具体而言，将基线架构记为权重为 W 的 f ，则时间 t 处的标准下一词元预测损失可写为：

$$\ell_t(W) = \text{CE}(f(x_{t-1}; W), x_t). \quad (1)$$

我们在测试时按顺序对每个 $t = 1, \dots, T$ 用梯度下降更新 W ：

$$W_t = W_{t-1} - \eta \nabla \ell_t(W_{t-1}), \quad (2)$$

其中 η 是学习率， W_0 是测试时的初始权重。最终，我们仅输出 $\hat{p}_{T+1} = f(x_T; W_T)$ 。我们在图 2 的左面板中展示了这种测试时训练形式：我们的示例基线仅使用向上的箭头，而测试时训练增加了向后和水平的箭头。

2.2 学习（在测试时学习）

我们现在考虑在同一示例范围内， W_0 ——（训练时）训练之后但在测试时训练之前的初始权重——是如何获得的。根据定义，我们的测试时训练损失 $\ell_t(W_{t-1})$ 也是下一词元预测任务的测试损失，该任务以 $x_0, \dots, x_{t-1}$ 为条件并试图预测 x_t 。因此，序列 $X = (x_1, \dots, x_T)$ 上的测试损失为：

$$\mathcal{L}(W_0; X) = \frac{1}{T} \sum_{t=1}^T \ell_t(W_{t-1}) = \frac{1}{T} \sum_{t=1}^T \text{CE}(f(x_{t-1}; W_{t-1}), x_t). \quad (3)$$

为了获得在测试时能产生低 $\mathcal{L}(W_0; X)$ 的 W_0 ，最直接的方法是在训练时对大量训练序列平均优化相同的损失。这种直接方法是端到端训练的一个例子，其中训练损失与测试损失相匹配。当测试时训练使用以这种方式训练的 W_0 时，我们称之为端到端测试时训练。

作为一个对比示例，考虑另一种方法，该方法天真地模仿静态模型的训练损失，而没有考虑到 W_0 将在测试时被更新：

$$\mathcal{L}_{\text{naive}}(W_0; X) = \frac{1}{T} \sum_{t=1}^T \ell_t(W_0). \quad (4)$$

这种方法不是端到端的，因为模型在训练和测试时的行为存在不匹配。因此，我们几乎无法保证 $\mathcal{L}_{\text{naive}}$ 的最小化器也会产生低的测试损失 L 。我们将这种方法称为 TTT-naive。它一直是动态评估文献中的主流方法 [72, 60]。

图 2 的右面板绘制了令牌级测试损失 ℓ_t ，在许多测试序列上取平均，对于 $t = 1, \dots, 128$ 。到目前为止，我们已经讨论了四种方法：具有完全注意力的变换器（橙色）、没有注意力的玩具基线（绿色）、TTT-naive（灰色）和 TTT-E2E（蓝色， $b = 1$ ；我们将在第 2.3 小节中介绍 $b = 16$ 的变体）；实验设置的详细信息见附录 A。虽然 TTT-naive 仅比玩具基线略好，但 TTT-E2E 的表现几乎与完全注意力一样好。特别是，TTT-E2E 可以有效地利用更多上下文来更好地预测下一个令牌，测试损失随时间下降证明了这一点。

对于基于梯度的优化，为端到端 L 计算 $\nabla L(W_0)$ 需要计算梯度的梯度，因为方程 2 中的更新规则本身包含梯度操作。幸运的是，现代自动微分框架可以以最小的开销高效地计算梯度的梯度 [13, 25]。一旦计算出 $\nabla L(W_0)$ ，我们就可以将其代入标准优化器。在元学习领域，对 L 的梯度步骤称为外循环，对 ℓ 的梯度步骤称为内循环。

当前版本的 TTT-E2E 在长上下文中的大型模型仍然存在两个问题。第一个是效率，因为我们的内循环有许多无法并行化的步骤。第二个是稳定性，因为内循环中的每个梯度步骤仅依赖于单个令牌，这很容易偶然导致梯度爆炸。下一小节将解决这两个问题。

2.3 小批量 TTT 和滑动窗口

上述两个问题源于同一原因：方程 2 执行的是在线梯度下降而非小批量梯度下降。给定规模为 T 的训练集，标准做法是将其划分为 T/b 个批次，每个批次大小为 b （假设可整除），并对每个批次执行一步梯度更新。与在线梯度下降（其中 $b=1$ ）相比，较大的 b 已知能同时提升并行性与稳定性。我们可以将小批量思想应用于 TTT 以获得相同益处。给定包含 $x_1, \dots, x_T$ 的（测试时）训练集，我们将方程 2 推广为：

$$W_i = W_{i-1} - \eta \frac{1}{b} \sum_{t=(i-1)b+1}^{ib} \nabla \ell_t(W_{i-1}), \quad (5)$$

对于 $i = 1, \dots, T/b$ （假设可整除），然后输出 $\hat{p}_{T+1} = f(x_T; W_{T/b})$ 。此外，为使（训练时）训练反映测试时训练的变化，我们也将方程 3 推广为：

$$\mathcal{L}(W_0; X) = \frac{1}{T} \sum_{i=1}^{T/b} \sum_{t=(i-1)b+1}^{ib} \ell_t(W_{i-1}). \quad (6)$$

注意， $b=1$ 时恢复方程 2 和 3。

然而，采用小批量 TTT 后，我们的模型在每个批次内再次成为二元模型，如图 2 中 $b=16$ 时的红线所示。例如，考虑包含 $x_1, \dots, x_b$ 的第一个小批量。由于每个预测 $\hat{p}_t = f(x_{t-1}; W_0)$ 都是使用 W_0 而非 W_{t-1} 进行的，我们观察到 $\ell_t(W_0)$ 随 t 增加而增大，因为 $\hat{p}_t$ 缺失了更多上下文，即直到 $t-1$ 的所有标记。这一观察在每个小批量内均成立，其中唯一不缺失上下文的预测是小批量内的第一个和第二个预测。这些递增的损失导致 TTT 的梯度步骤变差，最终表现为紫线相比蓝线的性能下降。

为解决这一问题，我们最终超越玩具示例，为架构增加滑动窗口注意力层。虽然玩具示例完全移除了自注意力层，但我们的主要方法仅将其限制在固定窗口大小 k 内。对于 $T=128K$ 的主要结果，我们将窗口大小 k 设为 8K，TTT 小批量大小 b 设为 1K。设置 $k \geq b$ 至关重要，这样模型才能在 TTT 有机会更新权重之前记住每个小批量内的上下文。

对基线架构的这一修改完成了我们的主要方法。接下来，我们介绍三个实现细节，然后考虑解码多个标记的任务。包含实现细节的主要方法如图 3 左图所示。

2.3.1 实现细节

实现三个细节对于达到我们报告的结果是必要的。我们将在第 3 节通过消融实验来证明这些细节的合理性。然而，它们仍可能只是我们实验设置的产物，在其他设置中不同的设计选择可能更为合适。

测试时训练（TTT）仅针对多层感知机（MLP）层。现代变换器（Transformer）由重复的块构成，每个块包含一个完整的注意力层（我们已将其替换为滑动窗口注意力）、一个多层感知机（MLP）层以及若干归一化层。在测试时训练（TTT）期间，我们冻结嵌入层、归一化层和注意力层，因为在内循环中更新它们会导致外循环不稳定。因此，多层感知机（MLP）层是测试时训练（TTT）期间唯一更新的层。

测试时训练（TTT）仅针对 1/4 的块。一般来说，当存储量更大时，压缩过程中丢失的信息更少。在我们的情况中，信息是上下文，存储是更新后的多层感知机（MLP）层。然而，更新更多层也意味着反向传播梯度需要更多计算。因此，我们在计算成本与随上下文长度扩展的能力之间有一个直观的权衡，我们将在第 3 节通过消融实验来说明这一点。根据消融实验，我们选择仅对最后 1/4 的块进行测试时训练（TTT），但其他实验设置，尤其是那些具有更长上下文的设置，可能需要不同的选择。

每个块包含两个多层感知机（MLP）层。测试时训练（TTT）的一个担忧是遗忘预训练期间学到的知识。我们采用最简单的方法来解决这一担忧。在进行测试时训练（TTT）更新的块中，我们添加一个静态的第二个多层感知机（MLP）层，作为预训练知识的“安全”存储。为了与基线进行公平比较，我们减小了整个网络（包括测试时训练（TTT）期间冻结的部分）中多层感知机（MLP）的隐藏维度，因此总参数数量保持不变。

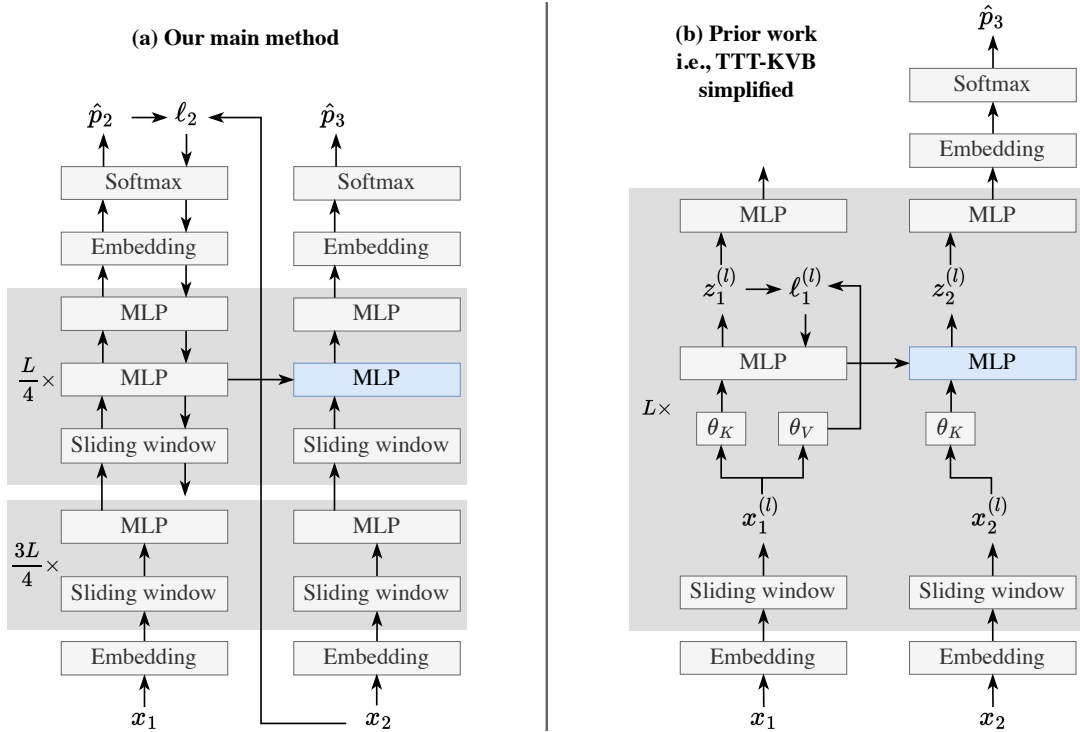


图 3. 遵循图 2 设置的计算图：给定 x_1 和 x_2 作为上下文，我们希望预测未知的 x_3 。左图：我们的主要方法，包含滑动窗口注意力层和第 2.3 小节讨论的实现细节。为了便于表示，我们的图示使用在线梯度下降（ $b = 1$ ）。最下方的向下箭头与下方的多层感知机（MLP）断开连接，因为梯度通过最后 $L/4$ 个块，但不会继续向下传递。右图：我们在第 2.4 小节中替代推导的第一步：先前工作中测试时训练键值块（TTT-KVB）的简化版本 [110, 87]。

2.3.2 解码多个标记

到目前为止，我们专注于预填充 $T + 1$ 个标记（包括 $\langle \text{BOS} \rangle$ 作为 x_0 ）然后解码单个标记的任务。现在我们考虑解码多个标记，我们的方法对此有一个自然的扩展：只有当解码的标记完全填满一个 TTT 小批量时，它才执行一次梯度步骤。例如，假设 T 能被 b 整除，那么 TTT 恰好用 T/b 个小批量耗尽预填充的标记。然后我们的方法在解码接下来的 b 个标记时不需要做任何特殊处理。之后，它对这个解码标记批次执行 TTT，然后继续使用更新后的权重进行解码。

2.4 另一种推导

本小节讨论我们主要方法的另一种推导，从基于键值绑定（Key-Value Binding, KVB）[87, 110] 的长上下文 TTT 的先前工作开始。关键步骤是用标准的下一标记预测损失替换他们逐层的重构损失，使 TTT 在测试时变为端到端（E2E）。理解第 3 节的结果不需要这个推导，但它提供了关于我们的方法如何与 RNN 文献相联系的额外洞见。

2.4.1 起点：键值绑定

基于将上下文压缩到模型权重中的相同思想，先前工作 [87] 使用 TTT 构建一个序列建模层，作为自注意力的直接替代品。

方法	损失	差异
SWA ($k = 8K$) 基线	2.827	TTT-KVB (Zhang 等人)
TTT-KVB 简化版	2.819	+0.001
TTT-E2E 全层多头	2.806	-0.013
TTT-E2E (ours)	2.805	-0.001

表 1。我们在第 2.4 小节中的替代推导从 TTT-KVB (Zhang 等人 [110]) 出发，经过三个步骤得到 TTT-E2E (本文方法)。这里，我们展示这些步骤的结果，以及作为参考点的 SWA 结果。我们认为 0.001 的差异低于统计显著性阈值。对于这里的结果，我们按照第 3 节的基本配方，使用默认超参数在 DCLM 上预训练并评估了 760M 模型。MH 表示多头。

自注意力通过将每个先前标记的键和值显式存储在缓存中来关联它们，而先前的工作提出通过测试时学习从每个键预测其值，将这些关联隐式地存储在模型中。这个学习目标后来被称为 KV 绑定，已成为许多流行 TTT 变体的核心组件，例如 MesaNet[98]、Titans[7] 和嵌套学习 [6]；线性注意力 [79, 55] 及其许多变体，如门控 DeltaNet[104]，也可以从这个角度推导出来。¹ 具体来说，给定第 l 层的输入嵌入 $x_t^{(l)}$ ，TTT-KVB 的基本形式在每个 $t = 1, \dots, T$ 上对以下损失 [87] 执行一步梯度下降：

$$\ell_t^{(l)}(W_{t-1}^{(l)}) = \left\| g\left(\theta_K^{(l)} x_t^{(l)}; W_{t-1}^{(l)}\right) - \theta_V^{(l)} x_t^{(l)} \right\|^2, \quad (7)$$

其中 g 通常是 MLP， $W_{t-1}^{(l)}$ 是 g 在上一个时间步后的权重， $\theta_K^{(l)}$ 和 $\theta_V^{(l)}$ 是外层参数，类似于 Transformer 中的键值投影矩阵。梯度步之后， g 使用更新后的权重生成输出嵌入：

$$z_t^{(l)} = g\left(\theta_Q^{(l)} x_t^{(l)}; W_t^{(l)}\right), \quad (8)$$

其中 $\theta_Q^{(l)}$ 也是一组外层参数。上述机制被称为 TTT 层。当在网络中使用时，每个 TTT 层都是一个独立的单元，拥有自己的损失和权重。在训练时，TTT-KVB 的外层循环与方程 6 中的 TTT-E2E 相同，所有外层参数，包括所有 TTT 层的 θ_K, θ_V 和 θ_Q ，一起优化。与第 2.3 小节中的 TTT-E2E 类似，当 TTT(-KVB) 层前面有滑动窗口注意力层时，可以有效地使用（内层循环）小批量梯度下降 [110]。这种混合架构作为我们推导的起点。

2.4.2 第一步：简化输出规则

首先，我们观察到方程 8 中的输出规则可以简化为：

$$z_t^{(l)} = g\left(\theta_K^{(l)} x_t^{(l)}; W_{t-1}^{(l)}\right), \quad (9)$$

如表 1 所示，几乎无损害。这一新的输出规则是一种简化，因为它重用了方程 7 中 g 的预测作为输出嵌入，而不是用更新后的权重和单独的输入再次调用 g $\theta_Q x_t$ 。直观上，如果滑动窗口注意力已经提供了足够的局部上下文，那么用更新后的权重调用 g 可能是不必要的，并且先前的工作已经论证了 θ_K 和 θ_Q 之间的分离也可能是不必要的 [59, 89, 108]。

¹ 如 [87] 所示，当 g 是线性模型时，TTT-KVB 恢复 DeltaNet[76]；当方程 7 中的梯度取 w.r.t. $W_0^{(l)}$ 而不是 $W_{t-1}^{(l)}$ 时，TTT-KVB 恢复线性注意力 [79, 55]；并且当 $\ell_t^{(l)}$ 使用非参数核估计器而不是参数模型学习时，TTT-KVB 恢复自注意力。

在图 3 的右侧面板中，我们展示了简化后的 TTT-KVB。与左侧面板中的 TTT-E2E 相比，有四个差异，其中后两个被视为实现细节：1. TTT-KVB 中的每个变换器块都有一个重构损失 $\ell^{(l)}$ ，而 TTT-E2E 在整个网络末端有一个单一的下一个标记预测损失 ℓ 。2. TTT-KVB 中的每个变换器块都有额外的外循环参数 θ_K 和 θ_V 。3. TTT-KVB 在每个变换器块中更新一个 MLP 层，而 TTT-E2E 仅在最后 1/4 的块中更新一个 MLP 层。4. 图中未显示，TTT-KVB 中更新后的 MLP 以与自注意力相同的方式分割成多个头，因此这些 MLP 比 TTT-E2E 中的常规 MLP 小得多。² 此外，这些 MLP 使用 LoRA[43] 进行更新，因此它们的有效容量甚至更小。

然而，图 3 也突出了这两种 TTT 形式之间的相似性：与 TTT-E2E 类似，TTT-KVB 可以从训练整个网络的角度来理解。首先，有一个通过整个网络的前向传递，如所有向上的箭头所示。然后有一个反向传递，其贡献来自许多损失，方式类似于深度监督网络 [62]。然而，梯度在仅到达一个 MLP 层后就被停止，如每个块中单个向下的箭头所示。

2.4.3 关键步骤：测试时的端到端

这一推导的关键步骤是将 KVB 损失替换为下一词元预测损失，这意味着同时消除差异 1 和差异 2，因为没有了逐层重构损失， θ_K 或 θ_V 也就没有用处。这一步将我们引向一种称为 TTT-E2E 全层多头 (MH) 的中间方法，其中 MH 代表多头。如表 1 所示，替换损失显著提升了语言建模性能，基本达到了我们最终方法的水平。

这一中间方法在测试时已是端到端 (E2E)，因为其 (测试时) 训练损失恰好是词元级测试损失 ℓ_t 。此时，特别值得关注的是我们两条推导路径之间的对偶性。主要推导从通过下一词元预测的 TTT 出发，在测试时已是端到端，并在第 2.2 节中专注于通过元学习使其在训练时也端到端。而另一条推导则从 TTT-KVB 出发，在训练时已是端到端，并专注于通过下一词元预测使其在测试时也端到端。

2.4.4 最后一步：以更少计算获得更大状态

TTT (-KVB) 层常被视为一类 RNN 层 [87]，而 TTT-KVB 常被视为一种 RNN。³ 类似地，TTT-E2E 也可被视为一种 RNN，只不过它只有一个 RNN 层，因为整个网络在一次反向传播中完成更新。在 RNN 的三个组成部分中，我们在推导的第一步修改了输出规则，在第二步 (关键步骤) 修改了更新规则。现在我们来修改隐藏状态。

通常能提升 RNN 长上下文性能的一个关键因素是更大的隐藏状态，而这往往需要更多计算来更新。考虑我们的中间方法 TTT-E2E 全层多头。如果通过仅更新最后 1/4 的块来消除差异 3，那么我们在测试时节省了计算，但最终得到的状态更小。如果通过恢复为常规 MLP (而非带 LoRA 的多头 MLP) 来消除差异 4，那么我们以更多计算 (和内存) 为代价获得了更大的有效状态。

² 为什么多头多层感知机比常规的更小？首先考虑一个线性模型。如果输入有 D 个维度，并且我们有 H 个头，那么每个头的输入将有 D/H 个维度。因此，每个头的线性模型将有 $(D/H)^2$ 个参数，所有头合起来将有 D^2/H 个参数，所以更多的头会导致更少的参数。由于多层感知机只是线性模型的堆叠，我们也有类似的关系。³ 具体来说，每个测试时训练层的隐藏状态是模型 g 的权重，其更新规则和输出规则由方程 7 和 8 定义。

然而，当使用端到端损失时，这两个差异的权衡相当不成比例：为了更新块中的小型多头多层感知机，梯度需要通过其上方的大型多层感知机反向传播，更不用说为了进一步进行反向传播还需要下方的注意力层。鉴于准备上游梯度的沉重成本，更新更少的块、每个块包含更大的隐藏状态应该更具成本效益。事实上，我们的最终方法（端到端测试时训练），同时消除了这两个差异，与所有层多头端到端测试时训练相比，具有 $5 \times$ 倍更大的隐藏状态（760M 模型为 88M 对 18M）和一半的推理延迟（在 H100 上每 1K 令牌预填充为 0.0086 对 0.017 秒）。

这个更大的状态是否真的能提高性能？在表 1 中很难看出差异，因为这些实验仅在 8K 上下文长度下进行。在第 3.2 小节中，我们将通过消融更新的层数，研究状态大小在上下文长度扩展方面的效果。这个消融将清楚地表明，较小的状态会导致更差的上下文扩展。

3 主要结果

所有实验都可以使用我们公共仓库中提供的代码和数据集重现：<https://github.com/test-time-training/e2e>。

3.1 设置

鉴于本文的研究性质，我们的目标是在小规模 and 中等规模的最简单设置中进行实验，这些实验可以为大规模的生产级训练运行提供信息。一般来说，当今的大规模运行通常由两个或更多阶段组成 [24, 103, 65, 74]：

- 在包含多样化知识的通用数据集上进行短上下文长度的预训练。
- 通过在长序列数据集上进行微调来扩展上下文长度。为了逐步达到非常长的上下文（例如 1M），扩展通常进一步分解为多个阶段。

为简单起见，我们的训练运行仅包含两个阶段：在 8K 上下文长度下进行预训练，以及在最终上下文长度（最多 128K，取决于实验）下进行扩展微调。

数据集。对于预训练，我们使用 DCLM，具体为 DCLM-Baseline，这是 Common Crawl 的一个经过严格过滤的子集 [63]。鉴于 DCLM-Baseline 中有 3.8T 个标记，我们首先丢弃所有短于 8K（我们的预训练上下文长度）的文档，然后从剩余文档中随机采样以构建各种规模的训练集。⁴ 然而，DCLM 中长度超过 128K（我们扩展的最大上下文长度）的大多数序列质量较低。因此，对于微调，我们使用 Books [29]，这是一个用于长上下文扩展的标准学术数据集 [4, 66]。我们还使用 Books 的留出分区进行语言建模评估。

基本配方。我们对五种规模的模型进行了实验，参数范围从 125M 到 3B。我们在各种实验中的配置和训练超参数均源自附录 B 中详述的单一基本配方。总之，模型配置和预训练的基本配方取自 GPT-3 [14] 和 Mamba [32]；为了生成微调的基本配方，我们对具有全注意力的 Transformer 基线进行了网格搜索。

基线。我们将我们的方法与六个基线进行比较，这些基线代表了架构设计中最先进的方法。所有使用滑动窗口的基线都使用相同的窗口大小 $k = 8K$ 。

1. 全注意力变换器 [95]：采用上述模型配置。
2. 滑动窗口注意力变换器（SWA）[8]：将每个全注意力层替换为 SWA 层。我们在第 2.3 小节中的主要方法，未包含实现细节。

⁴ 我们丢弃所有短于 8K 的文档，以避免在不同文档被打包到同一训练序列时跨文档边界重置更新后的 MLP，因为重置会降低我们基础设施上的训练速度。

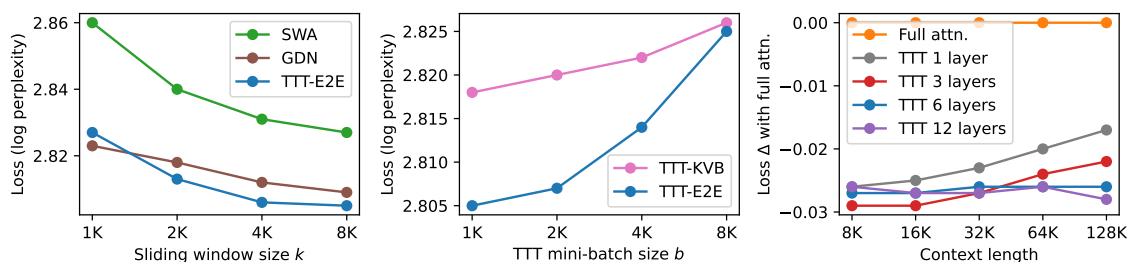


图 4. 对三个超参数的消融：滑动窗口大小 k 、小批量大小 b 以及 TTT 期间更新的层数；详见第 3.2 小节。根据这些消融的趋势，我们设置 $k = 8K$ ， $b = 1K$ ，并更新总层数的 $1/4$ 。损失 $\Delta(\downarrow)$ 。最右侧面板中的 y 值，与图 1 相同。其计算方式为（所报告方法的损失） - （全注意力变换器的损失），因此损失 Δ 为全注意力本身（橙色）是 $y=0$ 处的平坦线。GDN 代表门控 DeltaNet[105]。

也基于此架构。除窗口大小消融外，所有实验中窗口大小 k 均设为 $8K$ 。由于预训练上下文长度也是 $8K$ ，全注意力和 SWA 基线在扩展微调之前是相同的。3. 混合 SWA 与全注意力（5:1）[90]：重复五个 SWA 层后接一个全注意力层的模式，风格类似于 Gemma[90]。4. Mamba 2[21]：一种流行的 RNN，使用 Mamba 2 层与 SWA 层的混合；在 Nemotron-H[11] 中进行了大规模测试。5. 门控 DeltaNet[104]：一种流行的 RNN，扩展了 Mamba 2 和 DeltaNet[106]，并使用门控 DeltaNet 层与 SWA 层的混合；在 Kimi Linear[91] 中进行了大规模测试。6. TTT-KVB[110]：一种流行的 RNN，使用 TTT-MLP 层与键值绑定

（KVB）[87] 和 SWA 层的混合；也是我们在第 2.4 小节中的起点（未包含简化输出规则）。Titans[7] 和嵌套学习 [6] 采用了类似的构造。

我们在 JAX 中实现了基线 1-3 以及我们自己的方法。对于基线 4-6，我们使用作者提供的官方代码和配置，并在可能的情况下咨询他们以改进基线。我们对基线的改进在附录 C 中讨论。

3.2 超参数消融

为了帮助读者逐步建立对我们方法的经验直觉，我们从最简单的实验开始——对第 2.3 小节引入的超参数进行消融。对于所有消融实验，我们使用 760M 模型和基础配方。

滑动窗口大小 k 。除全注意力外，该超参数存在于所有方法中。因此，我们还对两个代表性基线进行了消融：SWA 和门控 DeltaNet。不出所料，如图 4 最左侧面板所示，更大的 k 能提升所有三种方法的性能，并且 TTT-E2E 对 k 变化的敏感度与基线相当。我们选择 $k = 8K$ 作为默认值，因为更小的 k 并不能显著改善运行时间。

测试时训练 (TTT) -E2E 与全注意力。窗口大小消融仅在 DCLM 上进行预训练，未在 Books 上微调，因此上述结果也在 DCLM 上评估。由于预训练上下文长度也是 $8K$ ， $k = 8K$ 的 SWA 恰好是全注意力，而 $k = 8K$ 的 TTT-E2E 则等同于在全注意力之上的 TTT-E2E。特别有趣的是，即使在完全注意力之上，TTT-E2E 也能将测试损失降低 0.018 ，并且随着 k 增大，TTT-E2E 与 SWA 之间的差异没有显著变化。这一观察表明，TTT-E2E 不仅仅是在弥补全注意力与 SWA 之间的差异；相反，当其他因素（如上下文长度）固定时，它产生了正交的改进。

测试时训练 (TTT) 小批量大小 b 。图 4 中间面板对 TTT 小批量大小 b 进行了实验，范围从 1K 到 8K。该超参数是 TTT 视角衍生方法所独有的，因此这里唯一能进行有意义比较的基线是 TTT-KVB。⁵ 与窗口大小消融类似，模型在预训练后在 DCLM 上评估。对于 TTT-E2E 和 TTT-KVB，我们观察到更大的 b 会显著损害性能。然而，小于 1K 的 b 也会显著损害硬件利用率和稳定性，以至于难以进行实验。因此，我们选择 $b = 1K$ 作为默认值。

无测试时训练 (TTT) 的修改架构。选择 $b = 8K$ 相当于完全不进行 TTT，因为我们的预训练上下文长度也是 8K。然而，不进行 TTT 的 TTT-E2E 和 TTT-KVB 都与全注意力的 Transformer 略有不同，因为这两种方法都略微修改了 Transformer 架构，如图 3 所示。那么，在没有 TTT 的情况下，这些修改是否仍然重要？图 4 表明答案是否定的。在没有 TTT 的情况下，TTT-E2E (2.825) 或 TTT-KVB (2.826) 的损失与全注意力 (2.827) 几乎没有差别。这一观察结果表明，架构设计在我们的方法中只起次要的辅助作用。

3.2.1 更新的层数

我们现在转向最重要的消融实验。如第 2.3 小节所讨论的，TTT 期间更新的层数控制着我们可以压缩上下文窗口信息的存储量。因此，我们从上下文扩展的角度研究其影响，并以图 1（左）的格式呈现该消融实验。具体来说，对于每个层数，我们在 DCLM 上预训练一个检查点，然后在 Books 上微调五个版本，每个版本对应一个上下文长度，因此最终结果在 Books 上进行评估。⁶ 我们尝试更新最后 1/2、1/4 和 1/8 的层。对于总共有 24 层的 760M 模型，这些比例分别对应最后 12、6 和 3 层。我们还尝试仅更新最后一层。从图 4 最右侧的面板中，我们观察到当仅更新 1 或 3 层时，我们的方法无法像全注意力那样随上下文长度扩展。当更新 6 或 12 层时，我们的方法确实能够扩展。然而，更新 12 层的性能与更新 6 层大致相同。因此，无论模型大小如何，我们始终更新最后 1/4 的层。

3.3 训练计算量的扩展

一般来说，训练计算量有两个维度：模型大小和训练 token 数量。我们研究了与全注意力和门控 DeltaNet 相比，我们的方法在这些维度上的表现，并将结果呈现在图 5 中。我们选择门控 DeltaNet 作为 RNN 基线的代表，因为它是最新的工作，且训练时间高度优化。

衡量训练计算量效果的一种常见做法是在预训练后立即在预训练数据集上进行评估，正如许多缩放定律论文中所见 [53, 40]。在图 5 的左侧面板中，我们遵循这一做法，在预训练后于 DCLM 上进行评估。但正如第 3.2 小节所讨论的，我们的窗口大小与预训练上下文长度相同，这使得我们的基线架构 SWA 等同于全注意力。这种等价性引发了一个担忧：上述做法可能无法揭示我们方法在没有全注意力情况下的真实行为。因此，我们还在微调后于 Books 上以 32K 上下文长度进行评估，如右侧面板所示。⁷

⁵Mamba 2 和门控 DeltaNet 有一个有些相似的超参数，称为块大小。然而，对于这些方法，块大小仅影响其硬件利用率而非输出，因此它们无法进行有意义的比较。⁶ 为了在 64K 和 128K 上下文长度下进行微调，这对于一个 760M 模型来说相当不寻常，我们不得不将基本配方给出的（外循环）批大小加倍。这一修改使我们能够对足够多的序列进行平均，从而使微调运行保持稳定。为了进行干净的比较，我们在本次消融范围内对所有上下文长度（包括较短的）的微调都采用了这一修改。⁷ 我们在 32K 处进行评估，因为较小模型（125M 和 350M）在更长上下文长度下的微调不稳定。

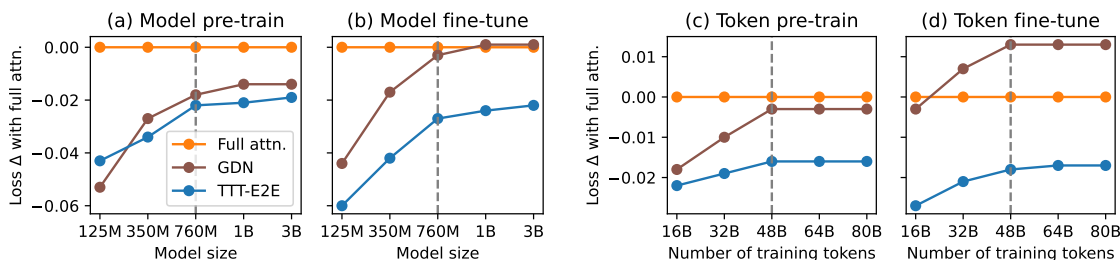


图 5. 训练计算量在两个维度上的缩放：模型大小（左）和训练标记数（右）；详见第 3.3 小节。总体而言，TTT-E2E 在较大训练预算下（虚线右侧）表现出与全注意力相似的趋势。我们报告了在预训练后于 DCLM 上以 8K 上下文长度（a, c）以及在相同上下文长度下微调后于 Books 上以 32K 上下文长度（b, d）的结果。损失 $\Delta(\downarrow)$ 即 y 值，与图 1 和图 4 中相同。最左侧面板中的图例在所有面板中共享。

对于模型规模的扩展，我们仅在本基础方案中的五种规模之间进行变化。对于训练标记数量的扩展，我们将模型规模固定为 7.6 亿参数，并改变预训练和微调的训练标记数量。具体而言，我们预训练的基础标记数量取自 Chinchilla 方案 [40]，而微调的基础标记数量为预训练的 5%，如附录 B 所述。我们实验了预训练和微调基础数量的最多 $5 \times$ 倍，并保持 5% 的比例不变。

在大预算下，与全注意力机制的趋势相似。我们在各面板中观察到相似的趋势：

- 在小计算预算范围内，TTT-E2E 相对于全注意力机制的优势随着训练计算量的增加而明显减小。
- 然而，在中等计算预算范围内，TTT-E2E 遵循与全注意力机制相似的扩展趋势，如蓝线保持相对平坦所示。尽管在模型规模扩展方面仍有小幅上升，但鉴于整体趋势，我们预计这种上升在更大模型上会消失。

对于模型规模的扩展，机制变化的边界大约在 7.6 亿参数。对于训练标记数量的扩展，该边界大约在 480 亿。我们在图 5 中用垂直虚线标出了这些边界。特别有趣的是，门控 DeltaNet 遵循与 TTT-E2E 相同的趋势。我们对此观察提供两种可能的解释：

- 如第 2.4 小节所述，我们的方法也可以解释为一种混合 RNN，类似于门控 DeltaNet。我们预期 RNN（具有固定大小隐藏状态的序列模型）在训练计算量扩展方面会呈现相似的趋势。
- 众所周知，与 RNN 相比，变换器在训练计算量不足时表现不佳 [53, 40]。我们的观察结果可以解释为全注意力基线在小计算量下的不足，而非 RNN 在大计算量下的不足。

总体而言，我们的实证观察强烈表明，在大预算生产运行中，TTT-E2E 在训练计算量扩展方面应产生与全注意力相同的趋势。

对分词器和数据质量的敏感性。在扩展性研究中，我们收集了关于分词器和数据质量影响的轶事观察，按时间顺序排列如下：

- 从 Llama 2 分词器（2023）切换到 Llama 3 分词器（2024），使我们在 3B 模型上相对于全注意力的优势提高了约 0.01。
- 从 SlimPajama（2023）[84] 切换到 DCLM（2024），使我们的方法在 48B 之后在训练 token 数量扩展方面产生与全注意力相同的趋势；使用 FineWebEdu（2024）[69] 的趋势也与全注意力相同。使用 SlimPajama 时，图 5 右侧面板中的曲线出现了小幅上升，类似于左侧面板中模型规模扩展的情况。

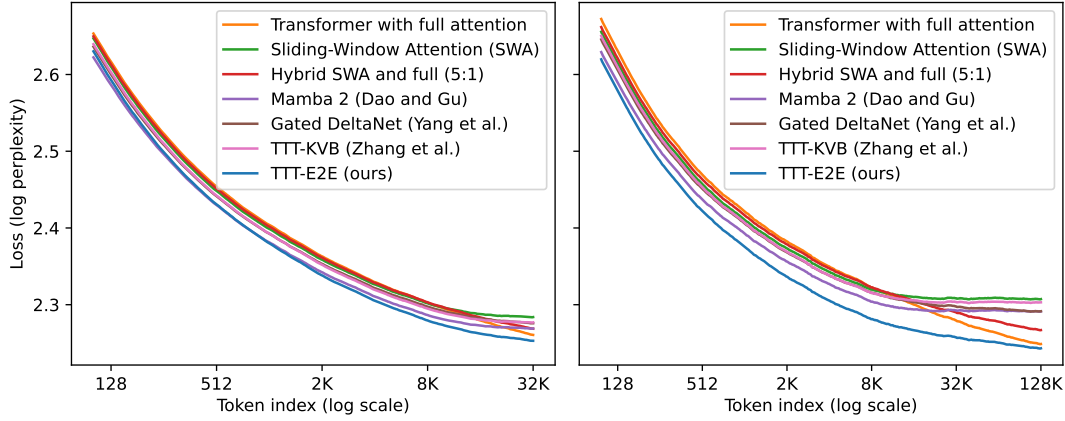


图 6. 按 token 索引划分的损失分解，上下文长度为 32K（左）和 128K（右），遵循与生成图 2 右侧面板相同的流程；详见第 3.4 小节。总体而言，TTT-E2E 是唯一在整个上下文长度内始终实现比全注意力更低损失的方法，其聚合优势主要来自较早的 token。

对这些影响的全面研究需要在各种分词器和数据集上重现图 5，这超出了本文的范围。尽管如此，我们的轶事观察仍可能为未来工作提供一个起点。一个特别有趣的方向是在自生成 token 上进行 TTT，这些 token 可以是当前 mini-batch token 的过滤或改写版本，也可以是对之前 mini-batch 的回顾。众所周知，RNN 中的门控机制可以保护隐藏状态免受虚假输入的影响，并更好地保留有价值输入中的信息 [39, 16]。我们相信，TTT 期间的自生成可以发挥类似的作用。

3.4 上下文长度扩展

我们在第一页的图 1 中展示了随上下文长度扩展的关键结果。这里，我们讨论这些实验的设置，并在图 6 中展示其中一些结果的分解。此外，附录中的图 9 直接绘制了图 1 中的损失值，而不是损失 Δ s。

对于图 1 中的实验，我们使用基本配方中的最大模型（3B）。我们还使用 $3 \times$ 个基本数量的标记进行预训练和微调。如前所述，预训练的基本数量取自 Chinchilla 配方，微调的基本数量为预训练的 5%。与之前的实验一样，我们在 DCLM 上预训练一个检查点，然后在 Books 上微调五个版本，每个上下文长度一个，因此最终结果在 Books 上评估。

3.4.1 按标记索引的损失分解

图 6 聚焦于两个上下文长度，32K 和 128K，并按标记索引分解图 1 中相应的结果；我们遵循第 2.1 小节中的相同过程来生成图 2 的右面板。具体来说，给定上下文长度 T ，对于每个 $t = 1, \dots, T$ ，我们绘制下一个标记预测任务的测试损失，该任务以 $x_0, \dots, x_{t-1}$ 为条件并尝试预测 x_t 。⁸ 因此，对于上下文长度为 T 的每种方法，其在图 1 中的测试损失是图 6 中相应曲线上所有损失的平均值。需要注意的是，32K 的分解不是 128K 分解的子集，因为它们是由两个不同的模型产生的。

⁸ 我们对每个基线重复此过程，但对于 TTT-E2E，我们的分解提供了另一种解释，如第 2.2 小节所述。我们在每个索引 t 处的测试损失也是测试时训练损失 $\ell_t(W_{t-1})$ 。由于 W_{t-1} 从未见过 x_t ，将 $\ell_t(W_{t-1})$ 与没有 TTT 的方法的测试损失进行比较是公平的。

Method	S-NIAH-1 (pass-key retrieval)					S-NIAH-2 (number in haystack)					S-NIAH-3 (UUID in haystack)				
	8K	16K	32K	64K	128K	8K	16K	32K	64K	128K	8K	16K	32K	64K	128K
Full attention	1.00	1.00	1.00	1.00	0.99	0.99	1.00	1.00	1.00	0.86	0.64	0.64	0.67	0.83	0.64
SWA	1.00	0.50	0.26	0.13	0.07	1.00	0.43	0.28	0.16	0.05	0.57	0.41	0.24	0.09	0.05
Hybrid SWA and full	1.00	0.93	0.88	0.69	0.21	1.00	1.00	0.99	0.89	0.29	0.63	0.56	0.32	0.17	0.06
Mamba 2 [21]	0.99	0.49	0.26	0.13	0.07	0.99	0.43	0.28	0.16	0.05	0.77	0.36	0.24	0.08	0.04
Gated DeltaNet [104]	1.00	0.50	0.26	0.13	0.07	1.00	0.43	0.27	0.16	0.05	0.91	0.45	0.23	0.07	0.03
TTT-KVB [110]	0.98	0.43	0.22	0.10	0.01	1.00	0.43	0.27	0.16	0.05	0.74	0.34	0.23	0.06	0.04
TTT-E2E (ours)	1.00	0.46	0.24	0.13	0.06	0.99	0.43	0.28	0.16	0.05	0.77	0.44	0.24	0.10	0.03

表 2. 不同上下文长度下的 S-NIAH 性能，最佳结果以粗体显示；详见第 3.5 小节。总体而言，具有完全注意力的 Transformer 显著优于其他方法，包括我们的方法，尤其是在长上下文中。这一观察结果，结合之前小节中的发现，支持了完全注意力的优势在于其近乎无损的召回率的直觉。

我们从图 6 的两个面板中得出以下观察结果：

- TTT-E2E 是唯一一种在整个上下文长度内始终实现比完全注意力更低损失的方法。
- 测试损失在上下文窗口末端附近，TTT-E2E 与全注意力的差异较小。TTT-E2E 相对于全注意力的累积优势主要来自较早的标记。

两个面板同时成立这两个观察结果，在某种程度上具有悖论性的趣味。作为第二个观察的一部分，左图中 TTT-E2E 与全注意力在 $t=32K$ ，上下文窗口末端附近的差异很小。在没有其他信息的情况下，人们甚至可能推测曲线会在更大的上下文长度（如 128K）处交叉。但这一推测是错误的，正如右图中第一个观察所断言的那样。128K 的分解图更像 32K 分解图的拉伸版本，而非推测的延续。鉴于 TTT-E2E 在图 1 中跨上下文长度保持对全注意力的相同优势，这种拉伸效应不应令人意外。

是什么让 TTT-E2E 对较早的标记具有优于全注意力的优势？请注意，这种优势甚至在 $t=1K$ 之前就已存在，此时 TTT 在第一个（内循环）小批量上执行第一次梯度步骤。换句话说，在 $t=1K$ 之前，TTT-E2E 和全注意力具有完全相同的计算图，仅在权重上有所不同。那么，为什么 TTT-E2E 的权重能产生低得多的损失？

这里有一个直观的解释：全注意力的权重必须准备好对上下文窗口中所有未来标记都表现良好。这样的任务可能非常困难，因为对所有可能的未来都表现良好会限制模型对任何特定未来表现良好的能力。但 TTT-E2E 的权重只需要对当前的小批量标记表现良好，因为 TTT 将为未来标记生成未来的权重。这个更聚焦的任务应该容易得多。事实上，正如我们将在第 4.2 小节中讨论的那样，TTT 的一个关键直觉是专注于当下。

3.5 大海捞针

如第 1 节所述，本文方法的动机是利用更长的上下文在语言建模中实现更优性能，而无需回忆每个细节。至此，我们主要关注无需详细回忆的评估。此处，我们考虑一种专为回忆设计的流行评估，称为“大海捞针”（Needle in a Haystack, NIAH）：模型需要在一段文本（干草堆）中检索目标字符串（针），该目标字符串因其与文本其余部分明显不相关而易于区分。具体而言，我们在 RULER [42] 的三个 NIAH 任务上评估所有在 128K 上下文长度下微调的 3B 模型。

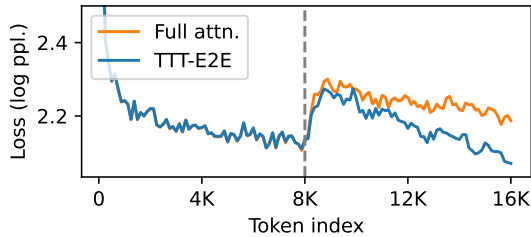


图 7. 使用 Qwen-8B 作为评估器解码长序列；详见第 3.6 小节。对于每种方法，我们用 Books 中的 8K 个标记预填充其上下文窗口，再解码另外 8K 个标记作为续写，然后按标记索引绘制 Qwen-8B 的损失，并在 512 个序列上取平均。虚线标记预填充与解码之间的边界。此图采用线性刻度而非对数刻度。

从表 2 中我们观察到，具有全注意力的变换器显著优于包括本文方法在内的其他方法，尤其是在长上下文场景下。这一观察结果结合前几小节的发现，支持了全注意力的优势在于其近乎无损的回忆这一直觉。这种优势源于自注意力设计的固有特性，即它会关注缓存中所有先前标记的键和值。相比之下，本文方法的核心机制是压缩，会忽略看似无关的细节，例如目标字符串。

3.6 解码长序列

至此，我们所有的评估都要求模型解码不超过十几个标记。如第 2.3 小节末尾所述，当解码的标记填满一个 TTT 小批量时，TTT-E2E 会在此批解码标记上执行一步梯度更新。这种测试时的“自训练”方法能否有效解码长序列？

在实践中，需要解码长序列的场景通常出现在指令微调之后或强化学习期间，例如模型生成长思维链时。因此，在没有上述两个阶段的情况下，以现实方式评估基础模型本质上具有挑战性。由于这两个阶段超出了本文的范围，我们尽最大努力评估第 3.4 小节中训练的 3B 基础模型。

对于图 7 中的评估，我们使用 Qwen-3-8B-Base [92] 作为评估器。由于我们的模型是在 Books 数据集上训练的，我们预先用 Books 中的 8K 个 token 填充其上下文窗口，再解码 8K 个 token 作为续写，然后按 token 索引绘制 Qwen-8B 在拼接后的 16K 序列上的损失（对数似然）。图 6 的 x 轴采用对数刻度，而图 7 采用线性刻度，便于我们比较预填充和解码阶段的趋势。该评估的更多细节见附录 D。

与之前的观察类似，在此有限评估中，TTT-E2E 的 Qwen 损失低于全注意力。此外，我们仔细检查了 ≈ 20 个生成文本样本，发现它们合理。两种方法的 Qwen 损失在预填充与解码的边界处急剧上升，随后逐渐回落。这一现象可能源于 Qwen 最初不熟悉被评估方法的生成风格，但随着其上下文窗口内生成内容的累积而逐渐适应。

3.7 计算效率

图 1 已展示我们的推理延迟，特别是预填充延迟，并与基线方法进行了比较。此处讨论预填充延迟的测量设置，并考虑计算效率重要的另外两个维度：解码与训练。特别指出训练延迟是当前实现的一个显著局限，并讨论两个潜在的改进方向。

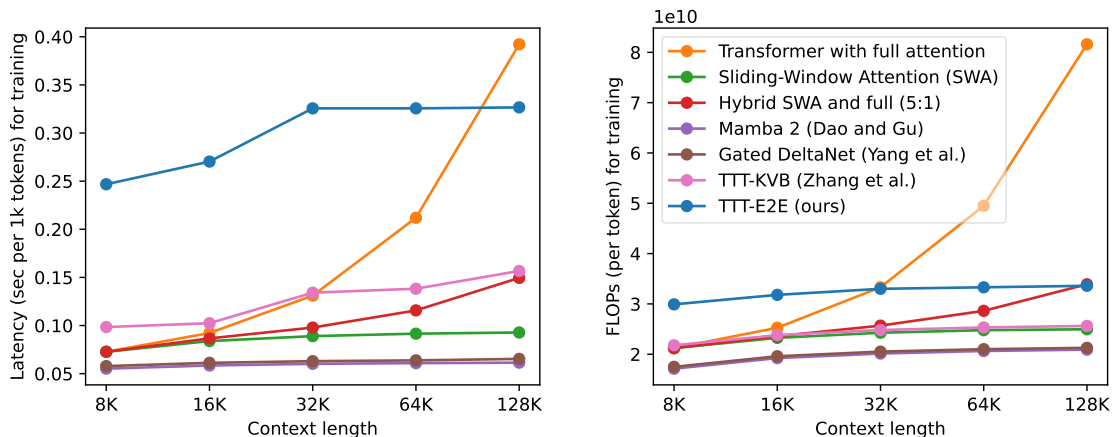


图 8. 训练效率，以 H200 上的延迟（左）和 FLOPs（右）衡量；详见 3.3 小节。总体而言，训练延迟仍是当前实现的一个显著局限。图例适用于左右两图。

预填充延迟的设置。对于图 1 右图中的每种方法，我们取左图中对应的 3B 模型，在单个 H100 上测量其预填充延迟。我们还采取了额外步骤来优化 PyTorch 基线的推理延迟，如附录 C 所述。遵循 Gated DeltaNet [104] 的做法，延迟实验在（外循环）批次中保持恒定的 token 数（128K）。例如，在 128K 上下文长度下，每个批次包含一个序列；在 8K 上下文长度下，每个批次包含 16 个序列。

测试时训练（TTT）-E2E 仅使用标准基础设施。在测试阶段，TTT-E2E 可以直接使用为训练常规变换器（Transformer）而优化的标准基础设施。具体而言，由于我们的隐藏状态采用常规多层感知机（MLP）层的形式，因此可以使用标准工具在多个图形处理器（GPU）之间进行分片，无需自定义核函数。相比之下，先前的工作必须将隐藏状态适配到 GPU 内部的单个芯片上，这极大地限制了其隐藏状态的大小。例如，TTT-KVB [110] 必须通过低秩适配（LoRA）来减小状态大小，而其他先前的工作，如 Mamba 2 [21] 和门控 DeltaNet [104]，则必须使用线性隐藏状态并编写自定义核函数以实现高效的内存输入输出。

解码延迟。正如第 2.3 小节末尾所讨论的，我们的方法在解码出的标记完全填满一个 TTT 小批量之前不会执行 TTT。因此，在达到完整批量之前，我们的解码延迟与带有滑动窗口注意力（SWA）的常规变换器相同。一旦获得完整批量，在解码下一批标记之前需要执行一步 TTT，而该 TTT 步骤的延迟与预填充阶段相同。总体而言，我们解码一个由多个批量组成的长序列的延迟就是上述两种延迟之和：滑动窗口注意力解码延迟与预填充延迟。由于这两者都容易获得，因此我们不单独报告 TTT-E2E 的解码延迟测量结果。

训练延迟的设置。我们的大部分训练是在 GB200 上进行的。由于我们的许多基线没有为 GB200（Blackwell）编写自定义核函数，因此我们在 H200（Hopper）上对训练延迟进行基准测试，以保证对基线的公平性。遵循我们的预填充协议，无论上下文长度如何，每个批量使用固定数量的标记（128K）。

训练延迟是一个局限性。在训练阶段，TTT-E2E 需要计算梯度的梯度，与训练常规变换器相比，这一过程的优化程度要低得多。如图 8 左图所示，在 128K 上下文长度下，我们的训练延迟比全注意力快 $1.2\times$ 倍，但在 8K 上下文长度下慢 $3.4\times$ 倍。由于大部分训练计算通常花费在短上下文的预训练上，因此我们当前实现的训练延迟仍然是一个显著的局限性。请注意，尽管每个标记的浮点运算次数保持不变，如右图所示，我们的延迟在 8K 到 32K 之间有所增长。这一趋势产生的原因是，我们必须将

时间维度上的梯度检查点数量增加 $\log(T)$ 倍，其中 T 是上下文长度。⁹

加快训练的方向。改善整体训练时间有两个方向：

- 我们当前的实现无法在训练时使用 cuDNN 闪存注意力 [20]，因为它不支持梯度的梯度。自定义注意力核函数将显著提高硬件利用率，并可能消除由时间梯度检查点引起的不良趋势。
- 我们认为 TTT-E2E 的训练可以从没有 TTT 的预训练变换器初始化——这是先前循环神经网络工作经常采用的技术 [54, 10, 99]。这种实用技术使 TTT-E2E 只占整体训练计算的一小部分，因此其训练延迟的负面影响最小。

我们将这些方向留给未来工作。

4 相关工作

4.1 持续学习

当今大多数人工智能系统在部署后保持静态，尽管世界在不断变化。持续学习的高层目标是使人工智能系统能够随着世界不断变化，类似于人类在一生中不断进步的方式 [36, 22]。

传统上，持续学习作为一个研究领域，主要关注从随时间逐渐变化的分布中学习 [68, 96, 33]。例如，可以每小时使用来自互联网的新知识更新聊天机器人模型，而模型的典型使用场景可能同时需要过去和现在的知识 [80, 56, 100]。更正式地说，在每个时间步，我们从当前分布中采样新的训练和测试数据，使用新的训练数据更新模型，然后在截至当前时间步的所有测试数据上评估模型。在这种设定下，大多数算法侧重于在学习当前知识时不遗忘过去 [75, 64, 57, 30]。

4.2 测试时训练

测试时训练的算法框架与持续学习具有相同的高层目标，但它侧重于人类学习在传统文献中的持续学习形式中表现突出的两个方面。

首先，每个人拥有独特的大脑，在其个人生活情境中进行学习。这种个性化的持续学习形式与例如每小时使用全球最新信息微调的聊天机器人模型截然不同。虽然这样的模型确实会随时间变化，但在任何给定时刻，它对每个用户和每个问题实例都是相同的。

其次，大多数人类学习发生在没有训练与测试边界的情况下。想想你今天早上的通勤。它既是“测试”，因为你确实关心今天早上能否到达工作地点；也是“训练”，因为你同时也在为未来的通勤积累经验。但在机器学习中，训练-测试划分一直是一个基本概念。

引入测试时训练的概念是为了实现人类学习的这两个特殊方面。训练通常涉及构建一个学习问题（如经验风险最小化）然后求解它。根据 [86]，测试时训练被定义为基于每个单独测试实例构建可能不同的学习问题的任何形式的训练。

默认情况下，诸如 JAX 和 PyTorch 之类的库在前向传播过程中保存中间激活，以便在反向传播过程中重用。然而，对于以 W 为隐藏状态的 TTT，这种默认设置会保存 $W_1, \dots, W_T$ ，这会占用过多内存。在这种场景下节省内存的标准技术是梯度检查点 [15]，它通常按层应用，但我们在训练过程中按时间应用它 [87, 25]。

这一概念在人工智能领域有着悠久的历史。自然语言处理中的一个著名例子是动态评估，由米科洛夫等人开创 [72]，并由克劳斯等人扩展 [60]，本文第 2.1 节在此基础上展开。在计算机视觉中，早期例子也已出现在人脸检测 [52]、视频分割 [73]、超分辨率 [83] 和三维重建 [70] 等应用中。接下来，本文讨论当今三种流行的测试时训练形式，重点阐述它们之间以及与历史例子的联系。

4.2.1 最近邻上的测试时训练：更大的有效容量

测试时训练的一种简单形式在 20 世纪 70 年代被称为局部加权回归 [85, 18]，在 20 世纪 90 年代被称为局部学习 [12]，在 21 世纪初被称为 KNN-SVM[109]：给定一个测试实例，在训练集中找到其最近邻，然后在这些邻居上训练（或微调）模型，再进行预测。这一过程可以显著增加模型的有效容量；例如，它允许线性模型拟合高度非线性的真实函数 [85]。

这种简单形式体现了测试时训练的一个关键直觉。在传统的机器学习观点中，模型一旦训练完成，在测试时就不再改变。因此，它必须准备好对未来所有可能的输入都表现良好。这项任务可能非常困难，因为对所有可能的未来都表现良好会限制模型对任何特定未来表现良好的能力。但实际只会发生一个未来。那么，为什么不在这个未来发生之后训练我们的模型呢？

最近，[35] 将这一思想扩展到现代语言模型，并观察到测试时训练后模型有效容量同样得到提升，[45] 通过更好的邻居选择策略进一步改进了这些结果。此外，[46] 表明，在训练集邻居上进行测试时训练对推理任务的强化学习同样有效，[5] 为视觉运动任务发展了相同的思路。

4.2.2 针对新实例的测试时训练：更好的泛化能力

随着当今模型规模日益增大，其能力往往不再受限于模型容量，而是受限于可用训练数据的数量，尤其是当模型需要泛化到分布外的新测试实例时。在这种情况下，使用静态模型为未来所有可能的测试实例（尤其是新实例）做好准备变得更加困难。然而，一旦给定具体的测试实例，我们便可以利用它来生成相关数据，进而用于训练 [88]。换言之，测试时训练的“邻居”不必来自训练集，它们也可以即时生成。

由于测试实例没有标签，使其对训练有用的一种方法是通过自监督，它为辅助任务（如掩码重建，例如 BERT[23] 和 MAE[37]）生成新的输入-标签对。虽然辅助任务与主预测任务不同，但通过共享表示，提升其中一个任务的性能有助于另一个任务。这种形式的测试时训练可以显著改善分布偏移下的泛化能力 [88, 28]。

最近，测试时训练已成为 AlphaProof[44] 的重要组成部分，该系统在 2024 年达到了国际数学奥林匹克银牌标准。针对每个测试问题，该系统首先通过提示语言模型生成由较简单问题组成的有针对性的课程，然后在生成的数据上进行强化学习。另一项近期工作 Akyurek 等人 [2] 发现，测试时训练对于 ARC-AGI 等少样本推理任务十分有效。他们的系统对测试问题中的少样本演示进行增强，然后执行监督学习。

4.2.3 序列上的测试时训练：更长的记忆

在目前讨论的所有测试时训练形式中，由于测试实例相互独立，模型在每次预测后都会被重置。然而，人类并不会不断重置自己的思维。我们对如何解决先前学习问题的记忆往往有助于解决当前问题，因为我们在世界中的经验更接近于相关数据序列，而非独立数据。

顺序应用（如视频和机器人技术）提供了一个弥合这一差异的试验场。例如，[34] 将 TTT 与自监督扩展到一种操作策略，其输入是机器人工作站的视频流，并发现无重置能带来更大的改进。最近，[101] 将同样的思想扩展到视频分割，使用仅用图像训练的模型。在这种情况下，TTT 可被视为将先前帧的上下文压缩到模型权重中，而无需学习如何学习，类似于我们在第 2.1 小节中方法的朴素版本。

测试时训练 (TTT) - 键值绑定 (KVB)。文本与视频一样，是一种序列形式。在第 2.4 小节中，我们讨论了 TTT-KVB 作为最相关的先前工作 [87, 110, 19]，其中包括 MesaNet [98]、Titans [7] 和嵌套学习 [6] 等变体。TTT-KVB 的流行有两个副作用：

- 由于 KVB 目标受自注意力启发，存储键和值，许多人认为长上下文 TTT 是关于记忆而非泛化。
- 由于 TTT(KVB) 层是自注意力层的直接替代品，许多人也将长上下文 TTT 视为一种架构设计方法。

我们的工作表明，长上下文 TTT 不需要记忆键和值之间的关联。此外，我们的方法纯粹是在持续学习问题的表述下推导出来的，对架构的改动最小。

4.3 快速权重与快速权重编程器

快速权重的一般思想是仅在最相关的数据上更新“快速”模型的参数，而不是在所有数据上更新“慢速”模型的常规做法 [94]。这一思想自 20 世纪 80 年代就已存在 [26, 38, 97]。由于最相关的数据通常可以包括测试实例本身，测试时训练可被视为快速权重的一种特例，更强调显式学习问题的表述。

快速权重编程器 (Fast Weight Programmers, FWPs) 的总体思想是在测试时通过一个“慢速”模型（作为编程器）更新快速权重，而该慢速模型本身更新频率较低，甚至完全不更新 [79]。在我们的方法中，内循环权重 W 可视为“快速”权重，外循环权重 θ 可视为“慢速”权重。因此，我们的方法可视为 FWPs 的一个特例 [58]。接下来，我们按相关度顺序简要回顾 FWPs 的相关文献。

Clark 等人 [17]。这项工作与方法论上我们的工作最为相关。给定一个具有全注意力的变换器 (Transformer) 基线，他们添加了一个多层感知机 (MLP) 层作为快速权重，其初始化作为慢速权重与模型其余部分一同训练。与我们的方法类似，他们通过对每个块（小批量）标记计算的下一个标记预测损失执行梯度步骤来更新快速权重。与基线相比，他们的方法显著改善了困惑度，但并未提升效率，因为其组合架构不具备线性复杂度。此外，他们的设计仅将快速权重添加到模型末端，而非与注意力层交错。在我们的实验中，交错被证明对于在更大基线上保持性能增益至关重要。尽管如此，我们认为 Clark 等人的工作具有宝贵的启发意义。更早的一项工作 [102] 也包含了类似想法的草图，但实验有限。

用于长上下文的 FWPs。许多解决长上下文问题的方法都源于 FWPs 文献。特别是，[79] (Schmidhuber, 1992) 一直是现代循环神经网络 (RNN) 层的主要灵感来源，例如线性注意力 [55, 77]、DeltaNet[76, 106] 和门控 DeltaNet[105] (我们的基线之一)。此外，一些关于长上下文测试时训练 (TTT) 的工作 [87, 110] (在第 4.2 小节中讨论) 由于 TTT 与快速权重之间的联系，也可视为 FWPs。值得注意的是，Irie 等人 [49] 中的一个实例使用 MLP 作为逐层快速权重来处理长上下文，早于 [87] 中的类似实例。

其他 FWPs。虽然上述快速权重程序 (FWP) 可以通过测试时训练 (TTT) 来解释, 但许多其他变体则不能。例如, [50] 设计了由自身编程的快速权重, [51] 利用图像作为快速权重构建图像生成器, [48] 将快速权重程序的连续时间扩展应用于时间序列分类, 而 [47] 和 [31] 则展示了更新规则的选择如何影响快速权重程序在形式语言识别任务上的表达能力。事实上, 所有具有某种门控机制的网络, 例如带有 SwiGLU 模块的变换器 [82], 也可以被视为快速权重程序 [32]。

4.4 学会学习

几十年来, 研究者们一直认为学会学习 (learning to learn), 也称为元学习 (meta-learning) 或双层优化 (bi-level optimization), 应当是智能的重要组成部分 [78, 9, 93, 61]。该领域中最相关的工作或许是模型无关元学习 (MAML) [27]。与本文工作类似, 模型无关元学习也有一个外层循环, 通过梯度的梯度来学习内层循环的初始化。模型无关元学习与本文工作的主要区别在于问题设定。具体而言, 其内层循环每次从整个数据集学习, 因此外层循环需要大量数据集。相比之下, 本文通过将语言建模问题转化为学会学习来解决该问题。原则上, 任何监督学习问题都可以转化为本文的问题表述。

5 结论

本文提出了 TTT-E2E, 一种用于长上下文语言建模的通用方法。原则上, 测试时训练可以应用于任何基线架构。在本文实验中, 该基线是带有滑动窗口注意力的变换器。将本文方法添加到该基线上, 会形成一种常见于生物记忆中的层级结构, 其中测试时更新的权重可解释为长期记忆, 而滑动窗口则作为短期记忆。本文认为这两类记忆将继续相互补充, 而更强的短期记忆形式将进一步改进组合方法。

作者贡献

本文陈述六位核心贡献者各自的贡献。

阿努夫·坦登 卡尔·达拉尔 (Karan Dalal) 主导了关于训练计算量扩展和上下文长度扩展的研究，开发了代码库，执行了最终实验，管理了我们的集群，并在本研究的各个方面发挥了核心作用，包括整体方向。

李新浩 (Xinhao Li) 开发并执行了大部分早期实验，包括玩具实验，与马塞尔共同开发了早期代码库，并为大规模训练贡献了功能。

丹尼尔·科切亚 (Daniel Koceja) 主导了一组项目中期实验，使团队对训练计算量扩展的研究更加清晰。

马塞尔·罗德 (Marcel Rød) 开发了大部分早期代码库，并提供了降低延迟的专业知识。

孙宇 (Yu Sun) 担任项目负责人，负责大多数日常事务的决策以及项目的整体方向。他提出了 TTT-E2E 的构想，设计了大部分早期实验，并为团队建立了实验规范。他还撰写了本文。

Yu Sun started the project in October 2024 together with Xinhao Li, Karan Dalal, and Daniel Koceja. Arnuv Tandon and Marcel Rød joined the project in November 2024. Karan Dalal and Daniel Koceja left from November 2024 through March 2025, and rejoined the project in April 2025. Marcel Rød left the project in May 2025. Xinhao Li and Daniel Koceja left in September 2025.

致谢

徐伟部分得到了美国国家科学基金会职业奖 IIS-2240014 的支持。田浩得到了天桥和陈丽乔基金会以及美国国家科学基金会职业奖 IIS-2338866、海军研究办公室 N00014-24-1-2609 和国防高级研究计划局合作协议 HR00112520013 的资助。本工作不一定反映政府的立场或政策，不应推断出任何官方认可。

作者感谢 Hyperbolic Labs 和 SF Compute 提供的 GPU 云服务，感谢 Voltage Park 的 Matt Behrens 及其服务团队在集群方面的帮助，感谢 Astera 的 Zacharie Bugaud 和 Alan Zheng 在每周会议中的讨论，感谢 NVIDIA 的 Jan Kautz 提供的一般研究支持，感谢刘壮在项目早期阶段的讨论，以及 Arjun Vikram、Gashon Hussein 和 Aaditya Prasad 的短期贡献。Arnuv 和 Karan 感谢 Y-Combinator 的 Harj Taggar 在计算积分方面的支持。Yu 感谢宋林洋和张天元对草稿和结果的反馈，感谢 Mert Yuksekgonul 和 Chloe Hsu 在项目各个阶段的支持，以及他的博士导师 Alexei A. Efros 和 Moritz Hardt 多年前的许多洞见，这些洞见最终成为本文的一部分。

参考文献

- [1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [2] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv e-prints*, pages arXiv–2411, 2024.
- [3] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. Learning to learn by gradient descent by gradient descent. *Advances in neural information processing systems*, 29, 2016.
- [4] Authors Guild. You just found out your book was used to train ai. now what?, 2023. Accessed: 2024-06-24.
- [5] Marco Bagatella, Mert Albaba, Jonas Hübötter, Georg Martius, and Andreas Krause. Test-time offline reinforcement learning on goal-related experience. *arXiv preprint arXiv:2507.18809*, 2025.
- [6] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. Nested learning: The illusion of deep learning architectures. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [7] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- [8] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [9] Yoshua Bengio, Samy Bengio, and Jocelyn Cloutier. *Learning a synaptic learning rule*. Citeseer, 1990.
- [10] Aviv Bick, Kevin Y. Li, Eric P. Xing, J. Zico Kolter, and Albert Gu. Transformers to SSMs: Distilling quadratic knowledge to subquadratic models. *arXiv preprint arXiv:2408.10189*, 2024.
- [11] Aaron Blakeman, Aarti Basant, Abhinav Khattar, Adithya Renduchintala, Akhiad Bercovich, Aleksander Ficek, Alexis Bjorlin, Ali Taghibakhshi, Amala Sanjay Deshmukh, Ameya Sunil Mahabaleshwarkar, et al. Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models. *arXiv preprint arXiv:2504.03624*, 2025.
- [12] Léon Bottou and Vladimir Vapnik. Local learning algorithms. *Neural computation*, 4(6):888–900, 1992.
- [13] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [14] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [15] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016.
- [16] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.
- [17] Kevin Clark, Kelvin Guu, Ming-Wei Chang, Panupong Pasupat, Geoffrey Hinton, and Mohammad Norouzi. Meta-learning fast weight language models. *arXiv preprint arXiv:2212.02475*, 2022.
- [18] William S Cleveland. Robust locally weighted regression and smoothing scatterplots. *Journal of the American statistical association*, 74(368):829–836, 1979.
- [19] Karan Dalal, Daniel Kocaja, Jiarui Xu, Yue Zhao, Shihao Han, Ka Chun Cheung, Jan Kautz, Yejin Choi, Yu Sun, and Xiaolong Wang. One-minute video generation with test-time training. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 17702–17711, 2025.
- [20] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [21] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

- [22] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE transactions on pattern analysis and machine intelligence*, 44(7):3366–3385, 2021.
- [23] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [24] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407, 2024.
- [25] Logan Engstrom, Andrew Ilyas, Benjamin Chen, Axel Feldmann, William Moses, and Aleksander Madry. Optimizing ml training with metagradient descent. *arXiv preprint arXiv:2503.13751*, 2025.
- [26] Jerome A Feldman. Dynamic connections in neural networks. *Biological cybernetics*, 46(1):27–39, 1982.
- [27] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.
- [28] Yossi Gandelsman, Yu Sun, Xinlei Chen, and Alexei A. Efros. Test-time training with masked autoencoders. *Advances in Neural Information Processing Systems*, 2022.
- [29] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 2020.
- [30] Spyros Gidaris and Nikos Komodakis. Dynamic few-shot visual learning without forgetting. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4367–4375, 2018.
- [31] Riccardo Grazi, Julien Siems, Arber Zela, Jörg KH Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. *International Conference on Learning Representations (ICLR)*, 2024.
- [32] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [33] Raia Hadsell, Dushyant Rao, Andrei A Rusu, and Razvan Pascanu. Embracing change: Continual learning in deep neural networks. *Trends in cognitive sciences*, 24(12):1028–1040, 2020.
- [34] Nicklas Hansen, Rishabh Jangir, Yu Sun, Guillem Alenyà, Pieter Abbeel, Alexei A Efros, Lerrel Pinto, and Xiaolong Wang. Self-supervised policy adaptation during deployment. *arXiv preprint arXiv:2007.04309*, 2020.
- [35] Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. *arXiv preprint arXiv:2305.18466*, 2023.
- [36] Demis Hassabis, Dhharshan Kumaran, Christopher Summerfield, and Matthew Botvinick. Neuroscience-inspired artificial intelligence. *Neuron*, 95(2):245–258, 2017.
- [37] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross B. Girshick. Masked autoencoders are scalable vision learners. *CoRR*, abs/2111.06377, 2021.
- [38] Geoffrey E Hinton and David C Plaut. Using fast weights to deblur old memories. In *Proceedings of the ninth annual conference of the Cognitive Science Society*, pages 177–186, 1987.
- [39] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [40] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.
- [41] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.
- [42] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

- [43] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- [44] T. Hubert, R. Mehta, L. Sartran, and et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025.
- [45] Jonas Hübottter, Sascha Bongni, Ido Hakimi, and Andreas Krause. Efficiently learning at test-time: Active fine-tuning of llms. *arXiv preprint arXiv:2410.08020*, 2024.
- [46] Jonas Hübottter, Leander Diaz-Bone, Ido Hakimi, Andreas Krause, and Moritz Hardt. Learning on the job: Test-time curricula for targeted reinforcement learning. *arXiv preprint arXiv:2510.04786*, 2025.
- [47] Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. Practical computational power of linear transformers and their recurrent and self-referential extensions. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [48] Kazuki Irie, Francesco Faccio, and Jürgen Schmidhuber. Neural differential equations for learning to program neural nets through continuous learning rules. *Advances in Neural Information Processing Systems*, 2022.
- [49] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers. *Advances in Neural Information Processing Systems*, 2021.
- [50] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, pages 9660–9677. PMLR, 2022.
- [51] Kazuki Irie and Jürgen Schmidhuber. Images as weight matrices: Sequential image generation through synaptic learning rules. *International Conference on Learning Representations (ICLR)*, 2022.
- [52] Vidit Jain and Erik Learned-Miller. Online domain adaptation of a pre-trained cascade of classifiers. In *CVPR 2011*, pages 577–584. IEEE, 2011.
- [53] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [54] Jungo Kasai, Hao Peng, Yizhe Zhang, Dani Yogatama, Gabriel Ilharco, Nikolaos Pappas, Yi Mao, Weizhu Chen, and Noah A. Smith. Finetuning pretrained transformers into RNNs. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2021.
- [55] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.
- [56] Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi, Gyuhak Kim, and Bing Liu. Continual pre-training of language models. *arXiv preprint arXiv:2302.03241*, 2023.
- [57] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- [58] Louis Kirsch and Jürgen Schmidhuber. Meta learning backpropagation and improving it. *Advances in Neural Information Processing Systems*, 34:14122–14134, 2021.
- [59] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- [60] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. In *International Conference on Machine Learning*, pages 2766–2775. PMLR, 2018.
- [61] Brenden M Lake, Tomer D Ullman, Joshua B Tenenbaum, and Samuel J Gershman. Building machines that learn and think like people. *Behavioral and brain sciences*, 40:e253, 2017.
- [62] Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, and Zhuowen Tu. Deeply-supervised nets. In *Artificial intelligence and statistics*, pages 562–570. Pmlr, 2015.

- [63] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, Etash Guha, Sedrick Scott Keh, Kushal Arora, et al. Datacomp-lm: In search of the next generation of training sets for language models. *Advances in Neural Information Processing Systems*, 37:14200–14282, 2024.
- [64] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.
- [65] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [66] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with blockwise ringattention. *arXiv preprint arXiv:2402.08268*, 2024.
- [67] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with blockwise ringattention, 2025.
- [68] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, pages 6467–6476, 2017.
- [69] Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. Fineweb-edu: the finest collection of educational content, 2024.
- [70] Xuan Luo, Jia-Bin Huang, Richard Szeliski, Kevin Matzen, and Johannes Kopf. Consistent video depth estimation. *ACM Transactions on Graphics (ToG)*, 39(4):71–1, 2020.
- [71] Dougal Maclaurin, David Duvenaud, and Ryan Adams. Gradient-based hyperparameter optimization through reversible learning. In *International conference on machine learning*, pages 2113–2122. PMLR, 2015.
- [72] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [73] Ravi Teja Mullapudi, Steven Chen, Keyi Zhang, Deva Ramanan, and Kayvon Fatahalian. Online model distillation for efficient video inference. *arXiv preprint arXiv:1812.02699*, 2018.
- [74] Team Olmo. Olmo 3, 2025.
- [75] Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. In *International conference on machine learning*, pages 1842–1850, 2016.
- [76] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.
- [77] Imanol Schlag, Tsendsuren Munkhdalai, and Jürgen Schmidhuber. Learning associative inference using fast weight memory. *arXiv preprint arXiv:2011.07831*, 2020.
- [78] Jürgen Schmidhuber. *Evolutionary principles in self-referential learning, or on learning how to learn: the meta-meta-... hook*. PhD thesis, Technische Universität München, 1987.
- [79] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- [80] Thomas Scialom, Tuhin Chakrabarty, and Smaranda Muresan. Fine-tuned language models are continual learners. *arXiv preprint arXiv:2205.12393*, 2022.
- [81] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024.
- [82] Noam Shazeer. Glu variants improve transformer, 2020.
- [83] Assaf Shocher, Nadav Cohen, and Michal Irani. “zero-shot” super-resolution using deep internal learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3118–3126, 2018.
- [84] Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. <https://cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama>, 2023.

- [85] Charles J Stone. Consistent nonparametric regression. *The annals of statistics*, pages 595–620, 1977.
- [86] Yu Sun, Xinhao Li, Karan Dalal, Chloe Hsu, Sanmi Koyejo, Carlos Guestrin, Xiaolong Wang, Tatsunori Hashimoto, and Xinlei Chen. Learning to (learn at test time). *arXiv preprint arXiv:2310.13807*, 2023.
- [87] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- [88] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning*, pages 9229–9248. PMLR, 2020.
- [89] Yi Tay, Dara Bahri, Donald Metzler, Da-Cheng Juan, Zhe Zhao, and Che Zheng. Synthesizer: Rethinking self-attention for transformer models. In *International conference on machine learning*, pages 10183–10192. PMLR, 2021.
- [90] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [91] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.
- [92] Qwen Team. Qwen3 technical report, 2025.
- [93] Sebastian Thrun and Lorien Pratt. Learning to learn: Introduction and overview. In *Learning to learn*, pages 3–17. Springer, 1998.
- [94] Tijmen Tieleman and Geoffrey Hinton. Using fast weights to improve persistent contrastive divergence. In *Proceedings of the 26th annual international conference on machine learning*, pages 1033–1040, 2009.
- [95] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [96] Gido M Van de Ven and Andreas S Tolias. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*, 2019.
- [97] Christoph Von Der Malsburg. The correlation theory of brain function. In *Models of neural networks: Temporal aspects of coding and information processing in biological systems*, pages 95–119. Springer, 1994.
- [98] Johannes von Oswald, Nino Scherrer, Seijin Kobayashi, Luca Versari, Songlin Yang, Maximilian Schlegel, Kaitlin Maile, Yanick Schimpf, Oliver Sieberling, Alexander Meulemans, et al. Mesanet: Sequence modeling by locally optimal test-time training. *arXiv preprint arXiv:2506.05233*, 2025.
- [99] Junxiong Wang, Daniele Paliotta, Avner May, Alexander M. Rush, and Tri Dao. The mamba in the llama: Distilling and accelerating hybrid models. *arXiv preprint arXiv:2408.15237*, 2024.
- [100] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application. *IEEE transactions on pattern analysis and machine intelligence*, 46(8):5362–5383, 2024.
- [101] Renhao Wang, Yu Sun, Yossi Gandelsman, Xinlei Chen, Alexei A Efros, and Xiaolong Wang. Test-time training on video streams. *arXiv preprint arXiv:2307.05014*, 2023.
- [102] Thomas Wolf, Julien Chaumond, and Clement Delangue. Meta-learning a dynamical language model. *arXiv preprint arXiv:1803.10631*, 2018.

- [103] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, Madian Khabsa, Han Fang, Yashar Mehdad, Sharan Narang, Kshitiz Malik, Angela Fan, Shruti Bhosale, Sergey Edunov, Mike Lewis, Sinong Wang, and Hao Ma. Effective long-context scaling of foundation models, 2023.
- [104] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.
- [105] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.
- [106] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.
- [107] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.
- [108] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. An attention free transformer. *arXiv preprint arXiv:2105.14103*, 2021.
- [109] Hao Zhang, Alexander C Berg, Michael Maire, and Jitendra Malik. Svm-knn: Discriminative nearest neighbor classification for visual category recognition. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, volume 2, pages 2126–2136. IEEE, 2006.
- [110] Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T Freeman, and Hao Tan. Test-time training done right. *arXiv preprint arXiv:2505.23884*, 2025.

Params.	Model configurations			Pre-training recipe			Fine-tuning recipe		
	Blocks	Dim.	Heads	Batch size	LR	Total size	Batch size	LR	Total size
125M	12	768	12	0.5M	3e-3	2.5B	1M	4e-4	125M
350M	24	1024	16	0.5M	1.5e-3	7B	1M	4e-4	350M
760M	24	1536	16	0.5M	1.25e-3	15B	1M	4e-4	750M
1.3B	24	2048	32	0.5M	1e-3	26B	1M	4e-4	1.3B
2.7B	32	2560	32	1M	8e-4	54B	2M	4e-4	2.7B

表 3. 我们的基础配方，其中 dim. 表示嵌入维度，批大小表示每个（外循环）批次中的标记数，总大小表示训练集中的标记总数。由于训练只有一个训练轮次，训练步数为总大小除以批大小。

玩具示例的配方

我们的玩具示例中有两种架构：具有全注意力的变换器和没有注意力的变换器。它们唯一的区别是后者移除了所有注意力层，因此参数更少。所有 TTT 方法都基于后者架构。两种架构都由两个变换器块组成。这些块基于基础配方中 125M 模型的块构建，嵌入维度减半（384），注意力头数减半（6）。

我们在上下文长度为 128 的 DCLM 上训练所有方法。训练令牌数遵循 Chinchilla 方案，外层循环批大小为 125K 个令牌。我们在与第 3 节相同的 DCLM 保留划分上进行评估。对于每种方法，我们在 $[2e-3, 6e-3]$ 范围内以 $0.5e-3$ 为增量搜索其最优学习率。最优学习率对于具有全注意力的 Transformer 为 $3e-3$ ，对于无注意力的 Transformer 及所有 TTT 变体为 $5e-3$ 。

B 基础方案

我们采用带有 QK 范数 [74] 的标准 Transformer 架构，以及 Llama 3 分词器 [24]。我们的基础方案如表 3 所示。它使用 GPT-3[14] 中的模型配置和预训练方案，并遵循 Mamba 2 论文 [21] 做了两处修改：我们使用 GPT-3 中峰值学习率的 $5 \times$ 倍，并将 1B 模型的批大小减半。这两处修改已被证明能改善全注意力基线。我们的学习率调度同样遵循 Mamba 2，对所有模型规模保持一致：在训练的前 10% 阶段，学习率按线性调度从 0 增加到峰值；在剩余训练阶段，按余弦调度衰减至 $1e-5$ 。

对于扩展微调，我们始终使用预训练词元数的 5%。同时将批大小加倍，以保证每批有合理数量的序列。为降低超参数搜索成本，我们仅对全注意力基线扫描峰值学习率。具体而言，针对 [760M、1B、3B] 中的每个模型规模及 [32K、128K] 中的每个上下文长度，我们扫描以下学习率： $[8e-5, 1e-4, 2e-4, 4e-4, 8e-4]$ 。我们发现单一选择 $4e-4$ 恰好在上列所有模型规模和上下文长度下表现最佳。因此，在基础方案中，我们对所有上下文长度和模型规模均采用该学习率。遵循 [66]，我们在微调开始时重新启动余弦调度。

我们使用 RoPE[59]，在 8K 上下文长度下预训练时采用 $\theta = 500K$ ，遵循 Llama 3[24]。对于扩展微调，我们按照标准做法 [24, 103] 为全注意力修改 RoPE θ 。遗憾的是，我们在大多数长上下文论文中未能找到 θ 的值。遵循 [67]，我们在 128K 时使用 $\theta = 10M$ ，并通过假设与上下文长度呈对数线性关系来选择 θ 的其他值： $\theta = 1M$ 用于 16K，2M 用于 32K，5M 用于 64K。

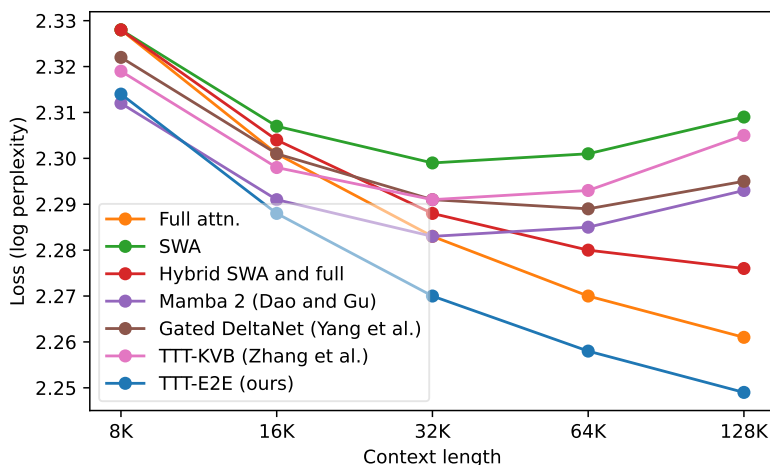


图 9. 随上下文长度的扩展。我们使用与图 1 左面板相同的结果，但直接绘制损失值而非损失 Δs 。更长的上下文长度改善了全注意力以及混合滑动窗口注意力与全注意力在所有上下文长度下的损失。在 32K 之前，它也改善了每种方法的损失。然而，对于滑动窗口注意力、曼巴 2、门控德尔塔网络和 TTT-KVB，超过 32K 后更长的上下文长度反而损害了损失。这一趋势的原因是，在扩展微调期间，更长的上下文导致每个（外循环）小批量中的训练序列更少，因此梯度具有更高的方差；同时，这些方法无法有效利用更长上下文的益处，因此更高方差的危害超过了益处。

C 基线的改进

我们对基线进行了两项改进。

延迟。我们的方法在 JAX 中实现，使用了最新的 cuDNN 注意力核函数。然而，所有基于 PyTorch 实现的循环神经网络基线最初使用的是 FlashAttention 2 核函数 [20]，其效率要低得多。为了改进基线，我们将它们的注意力层升级为使用最新版本的 FlashAttention 3 [81]。

查询键归一化。我们发现，在注意力层中对查询和键进行归一化（查询键归一化）使 TTT-E2E 的训练更加稳定，并且如先前工作 [74] 所建议的那样，也改善了变换器基线。因此，我们将其应用于其他具有滑动窗口注意力层的基线。在 760M 规模下，对门控德尔塔网络应用查询键归一化，在 8K 上下文长度的预训练中将其损失从 2.814 降至 2.809，在 32K 的扩展微调中从 2.691 降至 2.683。

D 解码评估的补充细节

在采样过程中，本文将 Softmax 函数的温度参数设为 1，并将前 p 个词汇的阈值 p 设为 0.95，遵循 [41] 的方法。

此外，众所周知，未经指令微调的基础模型在解码时往往会出现重复模式。为解决这一问题，本文采用 HuggingFace 工具包中用于生成的标准应用程序编程接口 (API) 来施加重复惩罚，并将其值设为 1.1。该值由众多 HuggingFace 用户推荐，经人工检查发现，它能使全注意力基线生成合理的文本。